Final project for CS3C: Benchmarking of different clonal lineage inference pipelines and approaches (AI-assisted version)

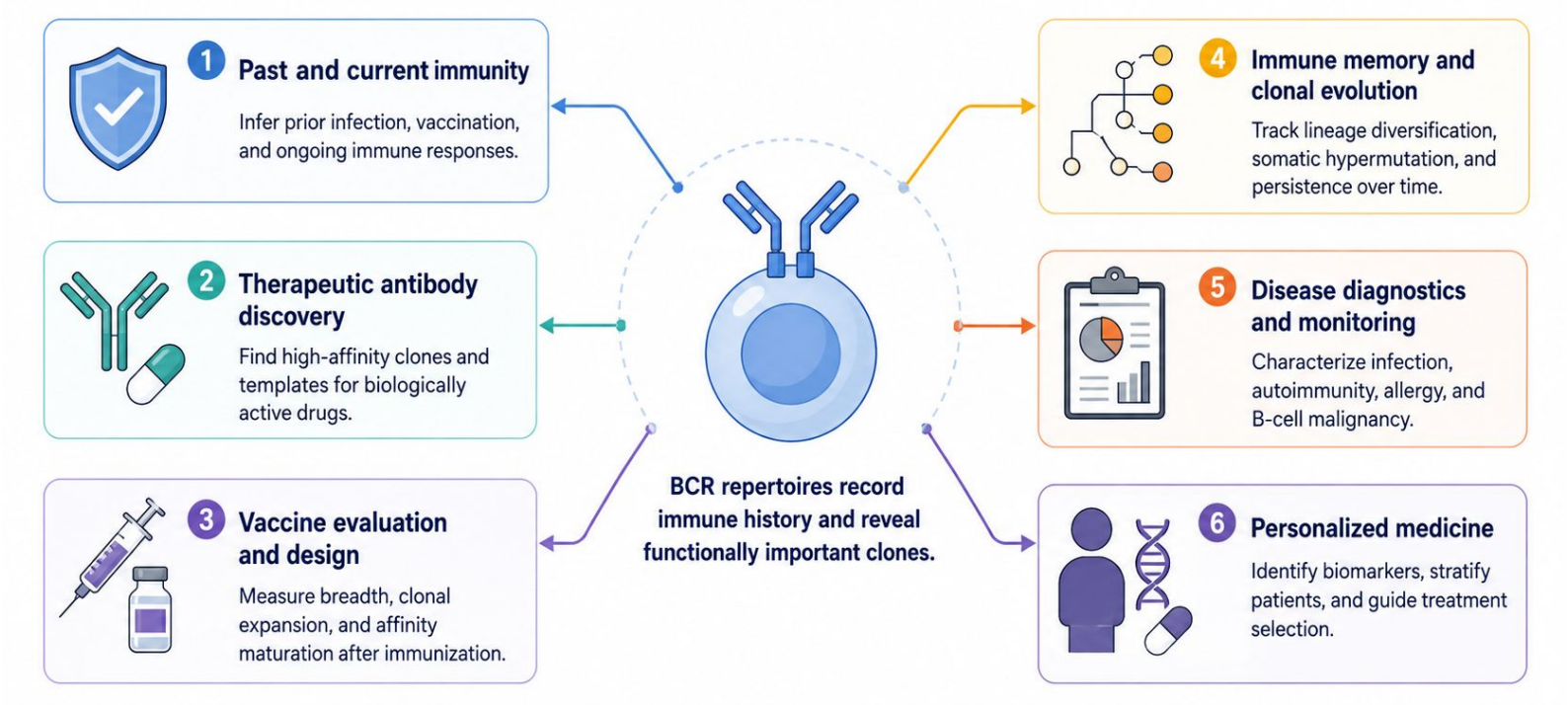
Valeriia Skatova

Project outline

1. Motivation: importance of clonal lineage reconstruction and its computational difficulty
2. Overview of existing approaches for clonal lineage reconstruction
3. Benchmarking results:
   a) Runtime
   b) Reconstruction accuracy, usability and consistency

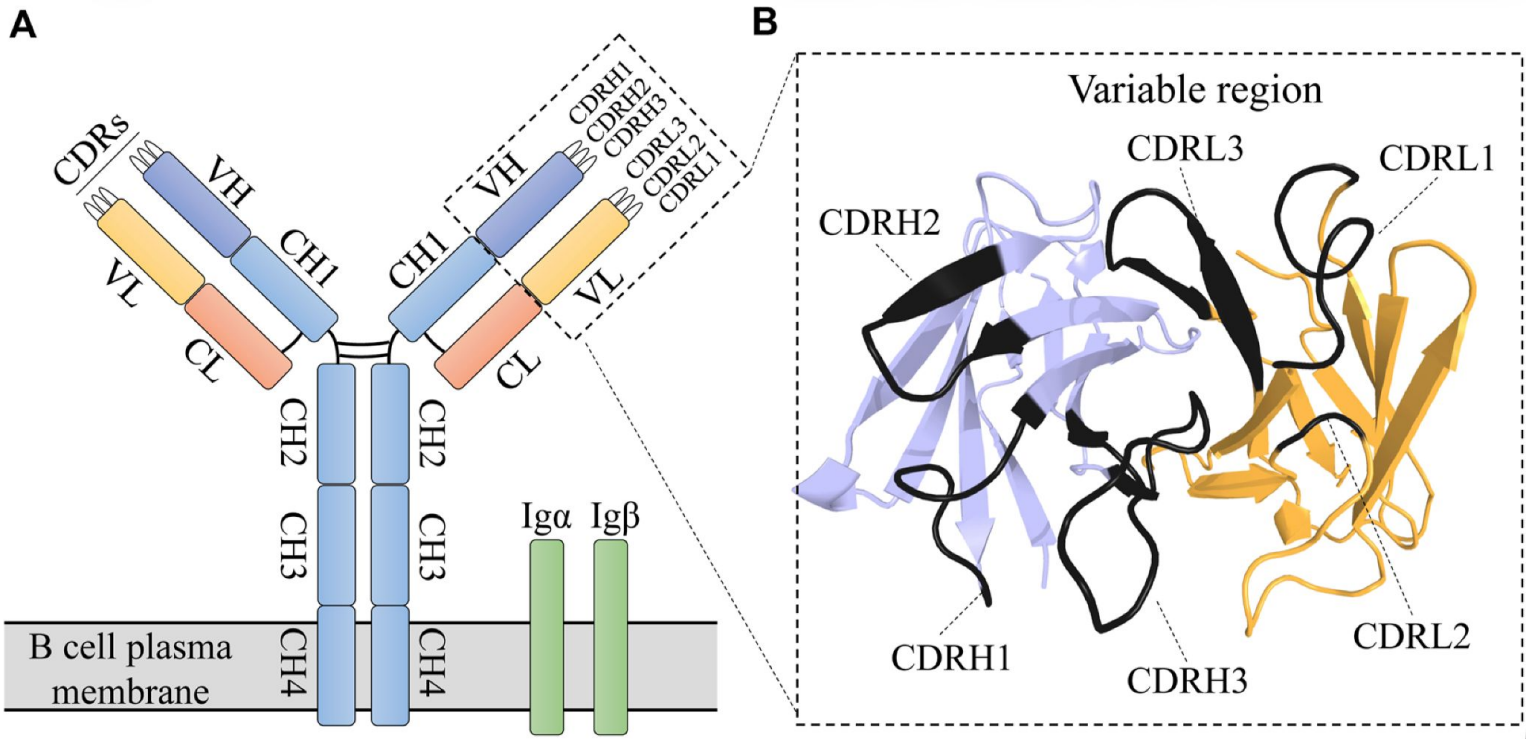
4. Conclusion

Motivation: why study B-cell receptors (BCR)?



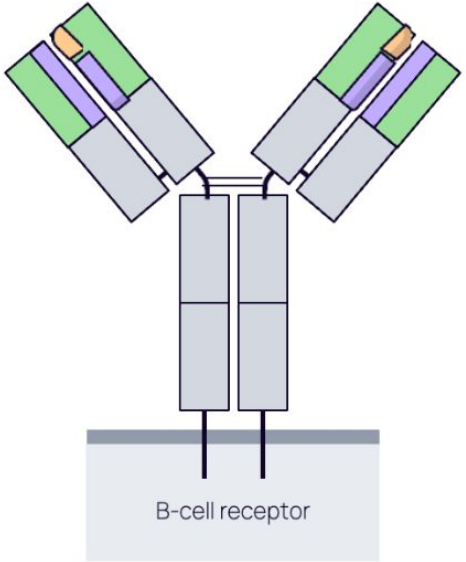
** This image was generated by Chat GPT*

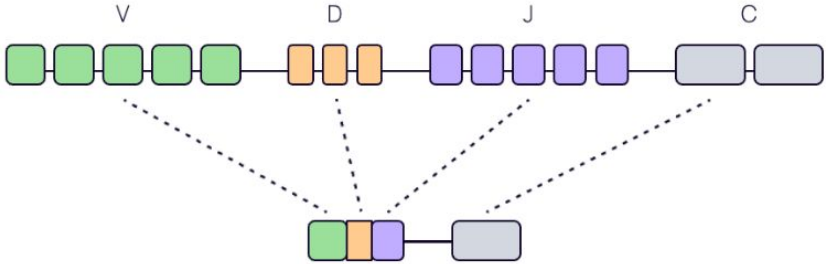
Motivation: what is B-cell receptor (BCR)?



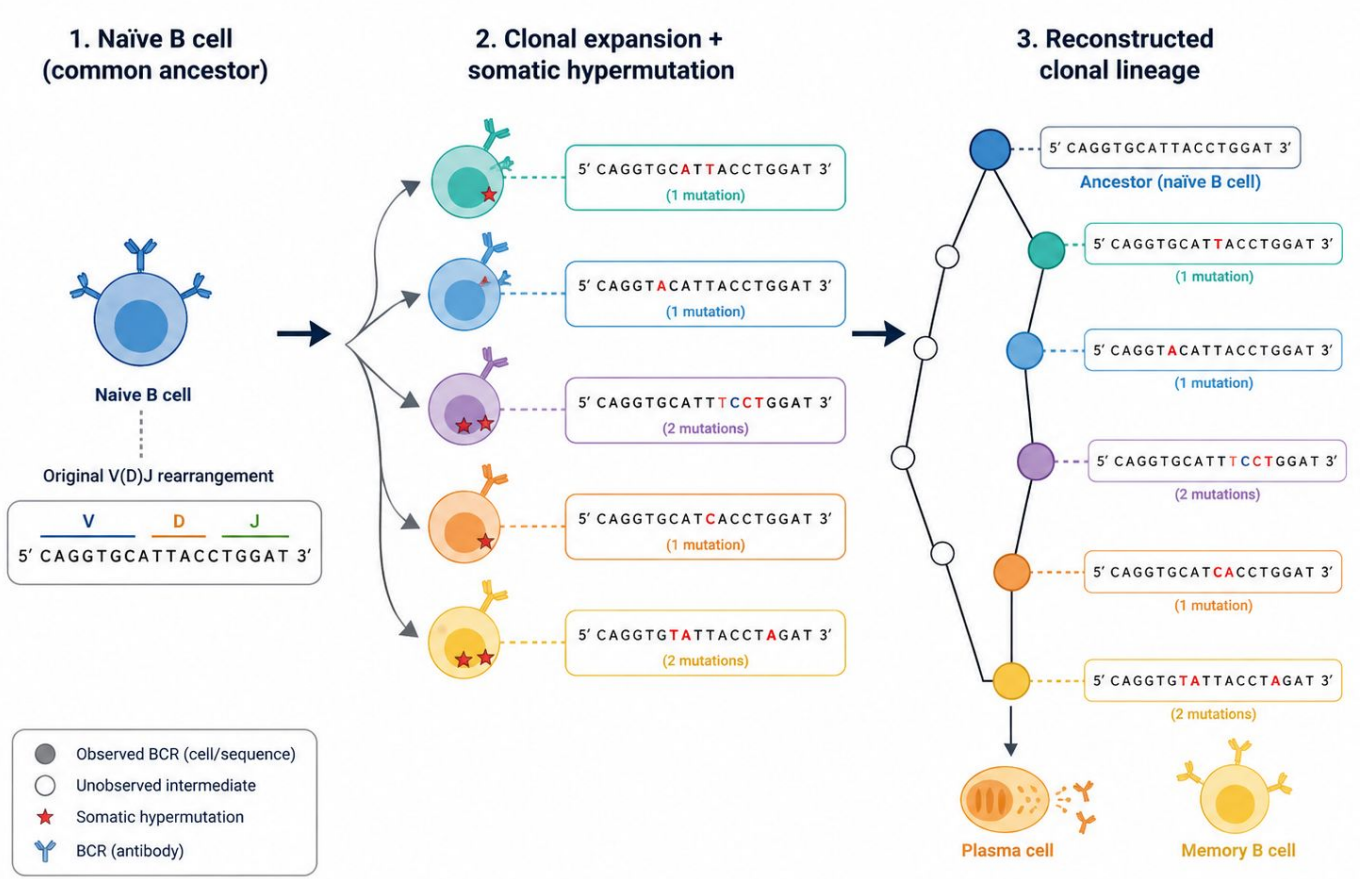
From Xu Z, Ismanto HS, Zhou H, Saputri DS, Sugihara F and Standley DM (2022) Advances in antibody discovery from human BCR repertoires. Front. Bioinform. 2:1044975. **doi: 10.3389/fbinf.2022.1044975**

Motivation: how BCRs are generated? VDJ recombination





Motivation: why clonal lineage reconstruction is non-trivial?

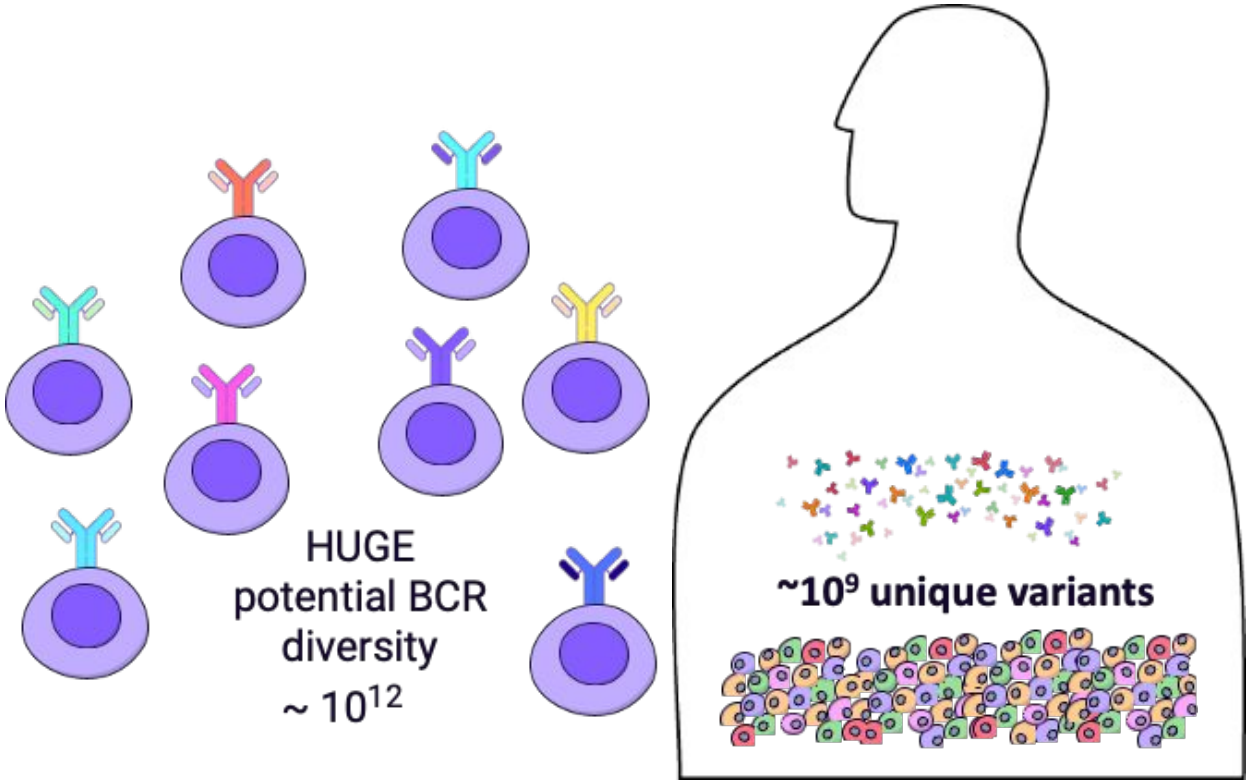


** This image was generated by Chat GPT*

Motivation: why clonal lineage reconstruction is non-trivial?

BCR diversity is coming from

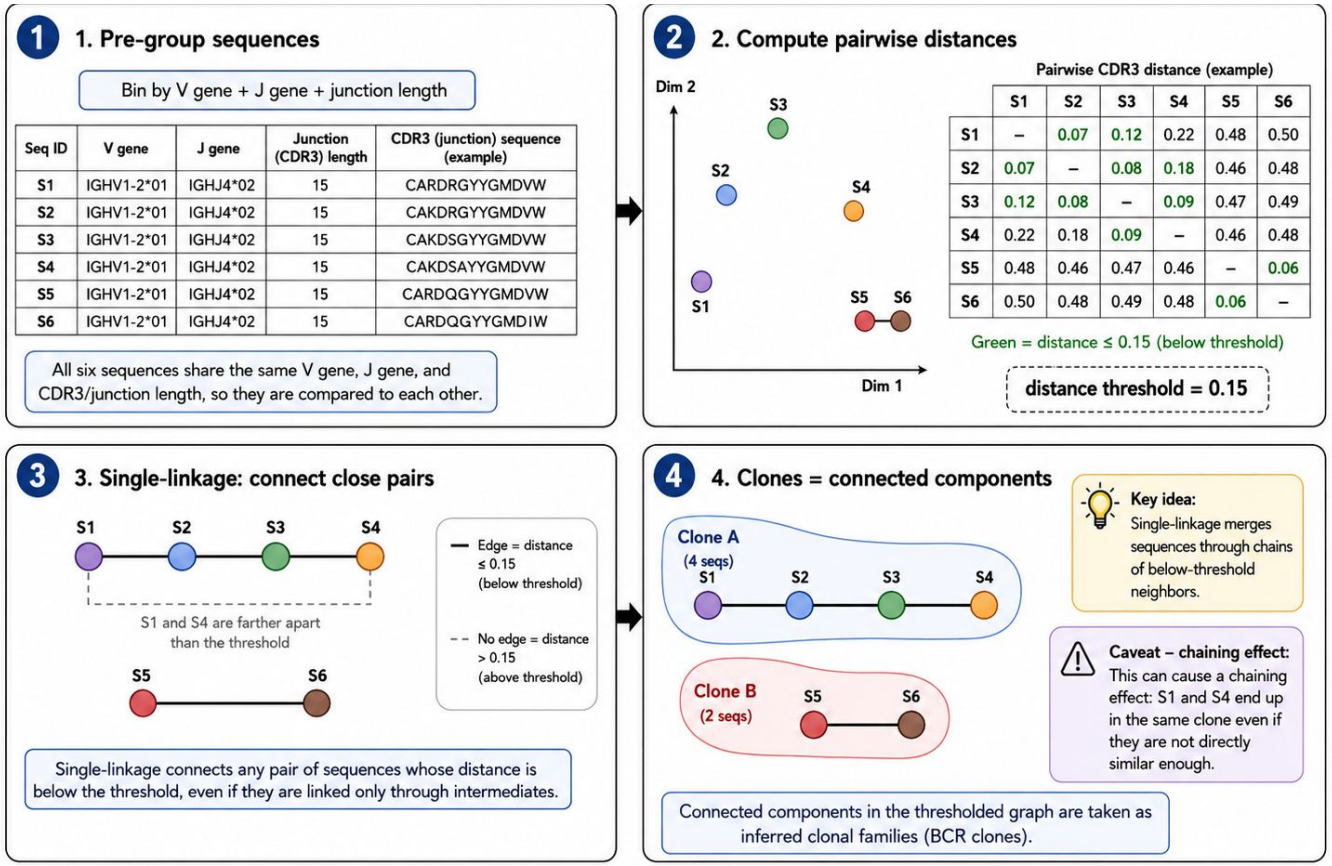
a) Semi-random VDJ recombination
b) Junctional insertions
c) Somatic hypermutation



Approaches to BCR clonal family inference for my benchmarking

Approach	**What is it**	**Language**
Single-linkage clustering with 80%/ 90% / 95% sequence similarity)	Flat R/python script	R/Python
fastBCR	R package	R
HILARy	full pipeline	Python
Scoper (Spectral and hierarchical clustering)	R package	R

1. V, J, CDR3 length grouping, hamming distance calculation and single-linkage clustering



Number of unique pairwise comparisons is n(n-1)/2 where n is number of sequences

** This image was generated by Chat GPT*

2. Scoper (Hierarchical)

Uses single-linkage clustering within V,J, CDR3 len groups based on Hamming distance with a fixed threshold of 0.85 sequence similarity

Implementation bottlenecks: pairwise Hamming distance calculation within groups O(n*n * length)

Non-algorithmic features to speed things up:

- -nproc option for parallelization

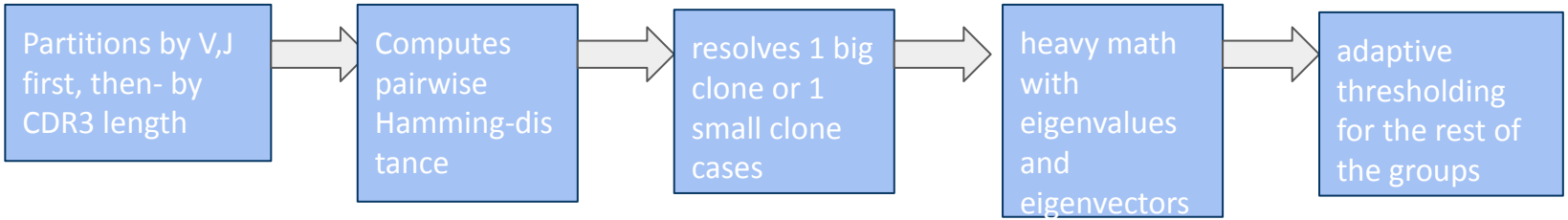
-installation as an R-package and clear documentation

-part of a widely used Immcantation framework

Obvious features that might slow things down:

-pre-required data formatting

2. Scoper (Spectral)



Implementation bottlenecks: pairwise Hamming distance calculation within groups O(n*n *length) + heavy math

Non-algorithmic features to speed things up:

- -nproc option for parallelization

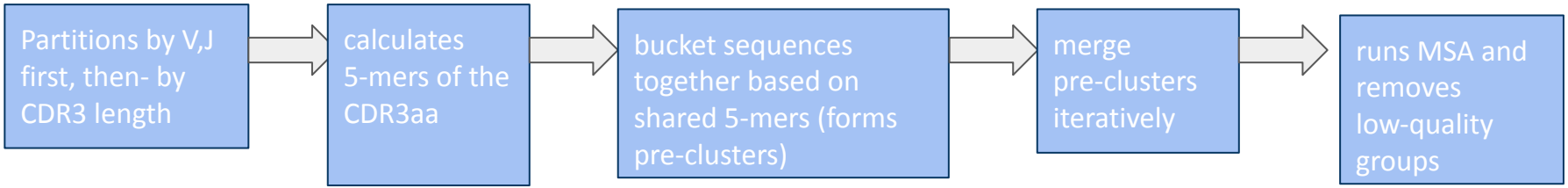
-installation as an R-package and clear documentation

-part of a widely used Immcantation framework

Obvious features that might slow things down:

-pre-required data formatting

2. fastBCR



Implementation bottlenecks: MSA (worst case O(length^{Nseqs})

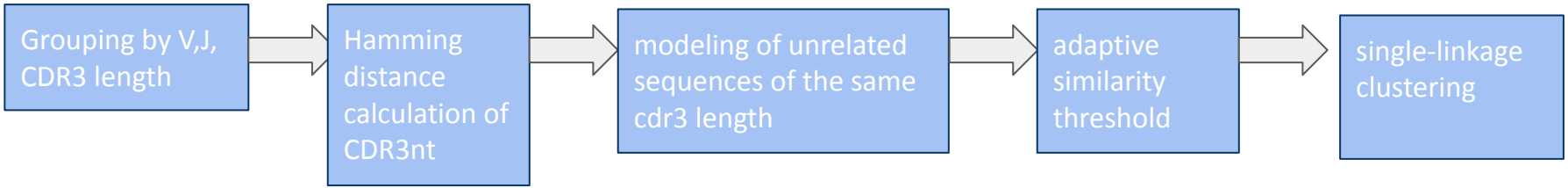
Non-algorithmic features to speed things up:

- installation as an R-package and clear documentation

Non-algorithmic features that might slow things down:

-pre-required data formatting

3.HILARy (high-precision inference of lineages in antibody repertoires)



Implementation bottlenecks: pairwise Hamming distance calculation within groups O(n*n*length), many intermediate lists for storage

Non-algorithmic features to speed things up:

-built-in option of parallelization thought –threads option (allows to run independent tasks on different CPUs)

-quick installation and clear documentation

Non-algorithmic features that might slow things down:

-pre-required data formatting

Benchmarking: input data

Real-worlds bioinformatics problem: Several donors have been tracked for several years, averaging for 20 to 30 samples per donor. The biological insights that can be obtained from this data requires me to run bcr clonal grouping on all samples coming from the same donor together.

The average size of the input is **1.5 mln sequences.**

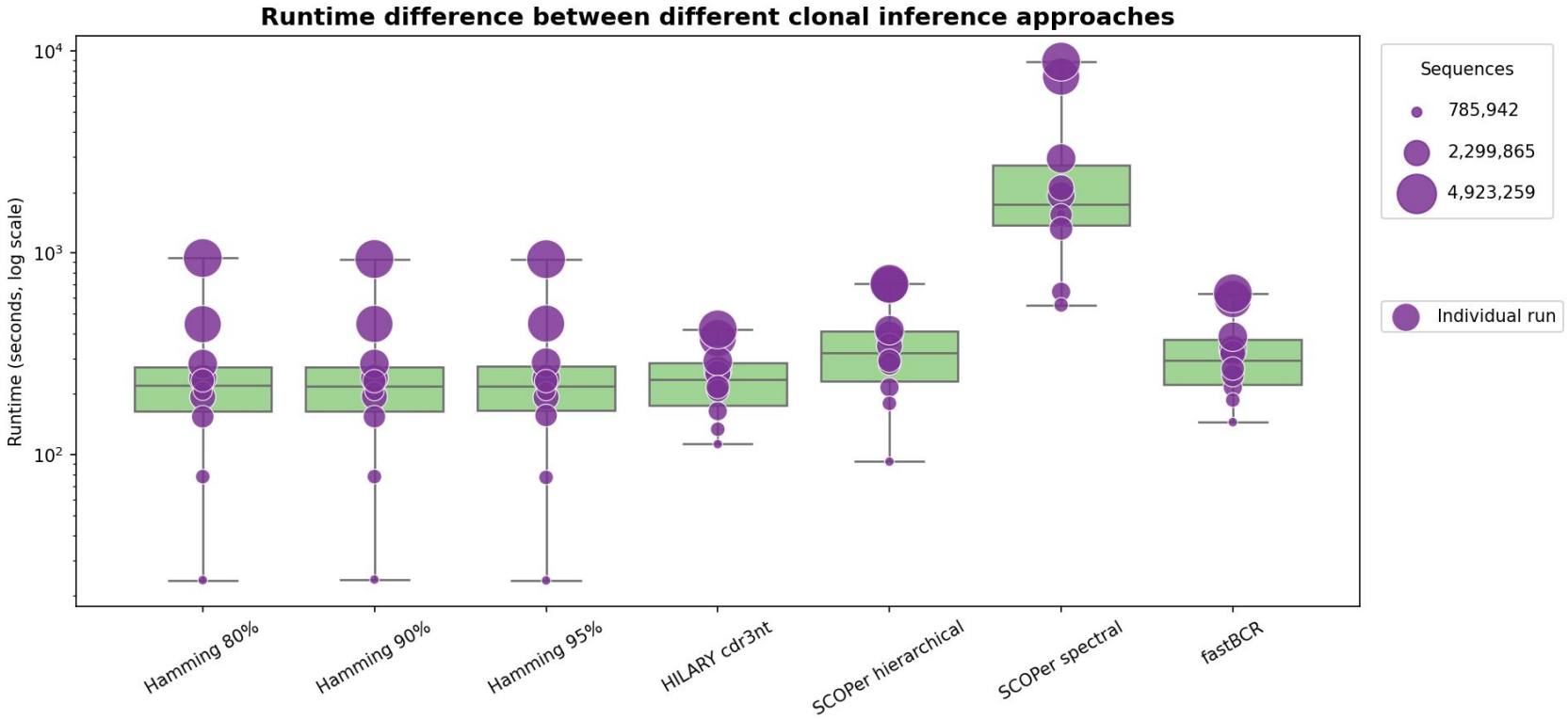
The maximum size of the input **7 mln sequences.**

The test run

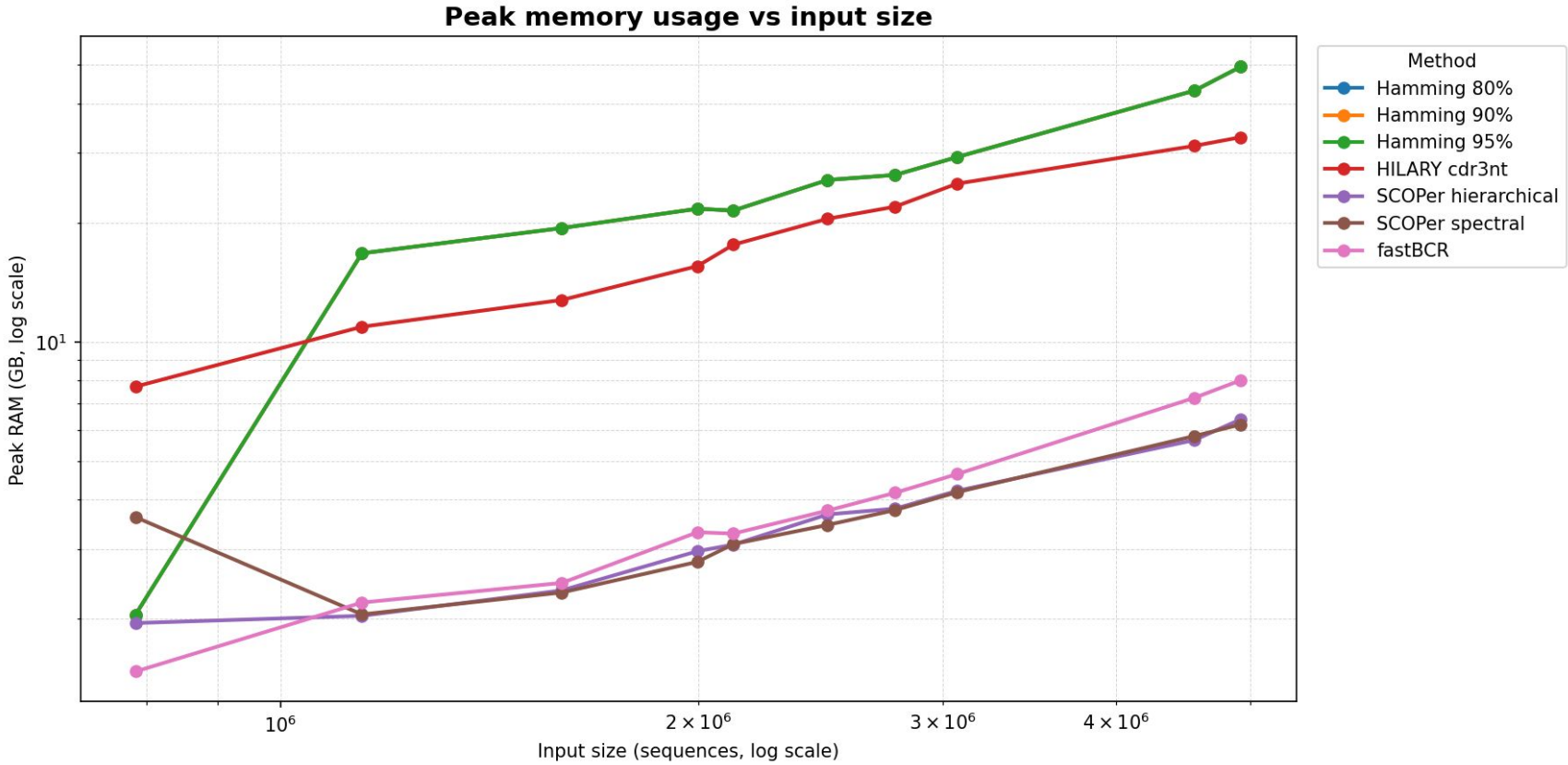
The smallest donor was chosen for the test run with 792k sequences

Method	Runtime	Peak RAM
Hamming 80,90,95%	~90 sec each	~2.1 GB RAM
Hillary (cdr3-method only)	113 sec	~7.9 GB RAM
Scoper Hierarchical	~110 sec	~2.0 GB RAM
Scoper spectral	~26 min	~3.7 GB RAM
fastBCR	~2.5 min	~1.5 GB RAM

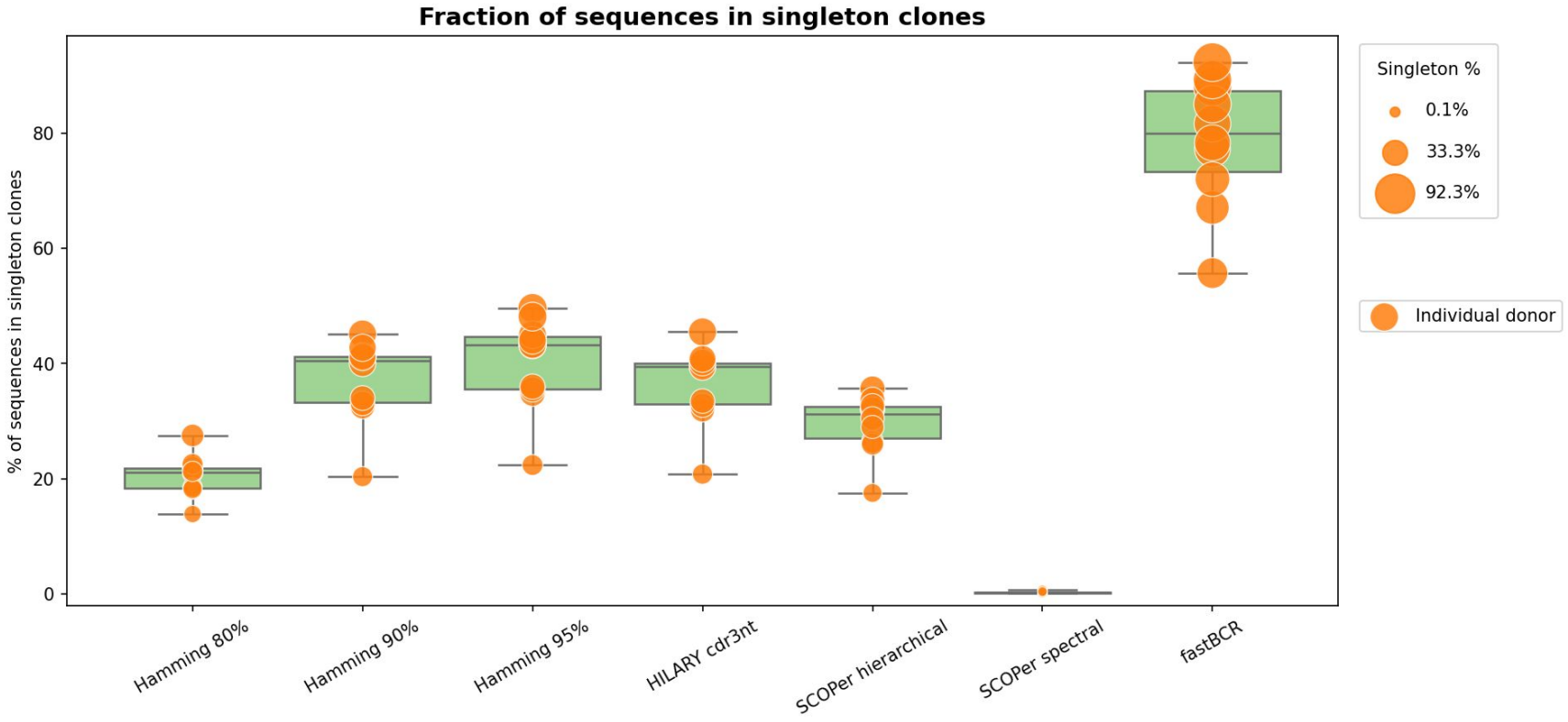
Benchmarking results: runtime



Benchmarking results: memory usage



Benchmarking results: output structure



Benchmarking results: logoplots for a long sequence

HILARY CDR3

fastBCR

Scoper Spectral

Hamming 90

Scoper Hierarchical

Benchmarking results: logoplots for a short sequence

fastBCR

Scoper Hierarchical

HILARY CDR3

Hamming 90

Conclusion

FastBCR - scalable, quite fast, produces adequate clones, although the speed is coming from losing all clonal groups lower than threshold (<=3) ;

HILARY cdr3 - less scalable because of memory inefficiency, fast only in “cdr3” mode, produces adequate clones

Scoper Spectral- very slow, over-merges clones

Scoper hierarchical - scalable, relatively fast, produces adequate clones, might under-clusuter

Python-script using single-linkage clustering + arbitrary similarity threshold- less scalable because of memory inefficiency, slow if the input is large, might under-clusuter

Opinion: I would chose fastBCR to look for **large public clones** (found in multiple people) or for the evidence of an acute response, often found in Plasmablasts. HILARY is a tool to go if i am interested in small, potentially medium sized groups , which often a part of a past memory response.